CJ Dropshipping

API connection manual
and the buyy catalog integration

buyy

1 October 2026

Contents

1 Purpose **2**

2 How a call is made **2**

3 Token **2**
3.1 Get a token 2
3.2 Refresh and logout 3

4 Rate limits **3**

5 API points **3**

6 Fetching one product **4**

7 Membership plans **5**

8 Orders, in brief **5**

9 The buyy integration **6**
9.1 What the shop shows 6
9.2 What is stored 6
9.3 How a sync runs 6
9.4 What this pass does not do 7

1 Purpose

This manual records two things. The first is how the CJ Dropshipping API 2.0 is connected to a website you own: the host, the token, the points budget, and how a single product is fetched. The second is how that catalog is wired into the buyy store without mixing it into the buyy product table, cart, or Paystack checkout.

The browser never calls CJ. A backend holds the API key, exchanges it for an access token, and serves pages from its own database.

Official reference: `https://developers.cjdropshipping.cn/en/api/start/development.html`.

The documentation site uses the `.cn` host. The API host uses `.com`. Calls are HTTPS, JSON, and UTF-8. URL paths are case-sensitive.

2 How a call is made

Every business URL has four parts:

`https://developers.cjdropshipping.com/api2.0/v1/{uri}`

Example:

`POST /api2.0/v1/authentication/getAccessToken`

Requests that carry a body send `Content-Type: application/json`. Every call except get-token and refresh-token sends the header `CJ-Access-Token`. Getting a token sends the API key in the JSON body and does not send that header.

Every response uses the same envelope.

Field	Meaning
`code`	Result code. `200` means success.
`result`	`true` when the call succeeded.
`message`	Human text. Do not branch on it.
`data`	Payload. Shape depends on the endpoint.
`requestId`	Id to log when a call fails.
`success`	`true` when the call succeeded.

Success is HTTP status 200 and either `code` equal to 200 or no `code` field. Anything else is a failure. Log `requestId` and `code`. By default an endpoint returns data only for the CJ account that owns the token.

3 Token

Create an API key in the CJ account (Apps, Install App, API, Add API, type API Key). The key looks like `CJUserNum@api@...`. Store it only on the server. Do not commit it and do not send it to the browser.

3.1 Get a token

`POST /api2.0/v1/authentication/getAccessToken` with a JSON body that contains `apiKey`.

Keep `openId`, `accessToken`, and `refreshToken`, plus both expiry dates, from the response. Do not hardcode a lifetime. Read the expiry timestamps from the response. Calling get-token again for the same account within 24 hours returns the same cached pair. The call itself is rate-limited like other endpoints.

3.2 Refresh and logout

Refresh with `POST/api2.0/v1/authentication/refreshAccessToken` and a JSON body that contains `refreshToken`. Replace the stored access token, refresh token, and both expiry dates.

`POST /api2.0/v1/authentication/logout` with header `CJ-Access-Token` expires both tokens. The next call must use the API key again.

Rules for the server:

- Cache the access token. Do not call get-token on every product request.
- If the access token is expired and the refresh token is still valid, refresh. If both are expired, call get-token with the API key.
- Keep the API key, both tokens, and `openId` on the server. The access token must not be returned to the front end.
- One CJ account is enough for one website. Each IP address may authenticate at most three CJ users.

4 Rate limits

Limit	Value
Per IP address	10 requests per second
Non-login interfaces	30 requests per second
Users per IP	3
Free, or sales level 0 or 1	1 request per second
Plus, or sales level 2	2 requests per second
Prime, or sales level 3	4 requests per second
Advanced, or sales level 4 or 5	6 requests per second

Get-token uses that same per-account rate. A tight loop over product pages hits the per-second cap before it hits the daily points cap. Space catalog calls out.

5 API points

CJ replaced the older daily call-count cap with points. For accounts registered on or after 1 June 2026 this is already in effect. For older accounts it takes effect on 1 July 2026. Until that date, the older request limits still apply to that account.

$$\text{daily total} = 50000 + \text{order conversion}$$

Order conversion is the maximum transaction amount of the last three months, in US dollars, times 100. The total resets at 00:00 UTC. Points also refill during the day at total/1440 per minute, and the available balance never rises above the daily total.

The base budget is therefore **50,000 points per day** before any order conversion is added. Calls that are not in the cost table do not spend points.

Endpoint	Points
`/product/listV2`, `/product/list`	50
`/product/query`, variant queries, most stock queries	10
`/product/stock/getInventoryByPid`	10
`/logistic/freightCalculate` and `freightCalculateTip`	10
`/product/queryProductsByImage`	1000

A buyy sync of one page is one list call (50 points) plus, for each of up to 20 products, a detail call and a stock call (10 + 10). That is about $50 + 20 \times 20 = 450$ points for one page. Five pages is about 2,250 points, still inside the 50,000 base, but the one-request-per-second cap on a Free account makes that sync take several minutes.

When points are not enough, CJ returns HTTP 429 with `code` 429 and a `pointsInfo` object (`usedToday`, `remaining`, `total`). Successful responses include the same object. Watch `remaining` during a long sync.

If the account has no transaction amount for 30 days in a row, API access is suspended. A reminder email goes out after 7 days with no transactions. Reactivate it from the API page in the CJ account.

6 Fetching one product

A keyword search is not how you load one known item. Keyword search is `GET /product/listV2` (`page`, `size`, `keyWord`). That returns a page of matches. Each row has an `id`. That value is the product id, `pid`.

To load one product you already know, call:

```
GET /api2.0/v1/product/query
```

Pass **one** of these query parameters. Do not send all three.

Parameter	Use it when
`pid`	You have the product id from `listV2` or from a previous sync.
`productSku`	You have the product SKU (the SPU), not the variant SKU.
`variantSku`	You have a variant SKU such as `CJxxxx-Black`.

Examples:

- `/product/query?pid=000B9312-456A-4D31-94BD-B083E2A198E8`
- `/product/query?productSku=CJJJJTJT05843`
- `/product/query?variantSku=CJJJJTJT05843-Black`

The detail payload includes the English name, images, description, `sellPrice` in US dollars, and a `variants` array. Each variant has a `vid`, a `variantSku`, and a `variantSellPrice`.

`vid` is the variant id you send when you later create an order against CJ's own catalog, with `orderFlow` set to 1. `sku` is the fallback when you do not have `vid`. If the detail call does not include every variant, call `GET/product/variant/query` with `pid`. Stock for the whole product is `GET/product/stock/getInventoryByPid`, also with `pid`.

`sellPrice` may be a single number or a range such as `8.13-8.62`. The currency is US dollars. The `unit` field is a selling measure such as pieces, not the currency.

7 Membership plans

These are the published monthly CJ account plans. Struck prices in the sheet are the previous list prices. Feature lists below are the rows visible on that sheet. Higher tiers also include the shared listing, employee, and store-authorization features of the free plan.

Plan	Monthly	Previous list	Who it is for
Free	$0.00	—	Basic features for all CJ users
Plus	$15.99	$59	New dropshippers
Prime	$19.99	$299	Growing dropshippers
Advanced	$59.99	$599	Advanced dropshippers

Prime is marked as the hot-sales plan on the sheet.

Plan	Included on the sheet
Free	5 daily sourcing requests. Free and unlimited listing. Unlimited employee accounts. Unlimited store authorization. Order monitoring. 24/7 support. Quality inspection.
Plus	$3 off $10 coupon, times 2. 5 VIP products. 50% off a course under $10. 10 extra daily sourcing requests. Free and unlimited listing. Unlimited employee accounts. Unlimited store authorization. Order monitoring.
Prime	$3 off $10 coupon, times 5. 10 VIP products. 15 recommended products. 50% off a course under $30. 20 extra daily sourcing requests. Free and unlimited listing. Unlimited employee accounts. Unlimited store authorization.
Advanced	$3 off $10 coupon, times 10. 20 VIP products. 25 recommended products. 50% off a course under $100. 50 extra daily sourcing requests. Free and unlimited listing. Unlimited employee accounts. Unlimited store authorization.

The API request rate follows the same names: Free is 1 request per second, Plus is 2, Prime is 4, and Advanced is 6. The 50,000 point base is separate from these subscription prices. Buying Plus, Prime, or Advanced does not, by itself, replace that points formula.

8 Orders, in brief

This manual does not turn on live fulfillment. The connection guide still defines the path, and it is recorded here so a later checkout change has a source.

When a customer pays the store, the backend may create a matching CJ order. Create Order V3 is `POST /shopping/order/createOrderV3`. For CJ's own logistics, set `shopLogisticsType=2`, leave `storageId` empty, set `logisticName` from a freight quote, and set `orderFlow=1` with each line carrying a CJ `vid` or `sku`. `fromCountryCode` is often `CN`.

The pay path after create is add-to-cart, confirm, save the parent order, then `payBalanceV2`, which spends CJ balance. Check `GET /shopping/pay/getBalance` first. Sandbox orders use `isSandbox=1` and do not spend real balance. Prove that path before any live pay call.

Webhooks need a public HTTPS callback. `localhost` is rejected. Verify `sign` as HMAC-SHA256 of the raw body, using the decimal `openId` as the secret, then Base64. Do not put `openId` in client logs.

9 The buyy integration

buyy is the Ghana fashion store. Its own catalog is priced in Ghana cedis and checked out with Paystack. CJ specials are a second catalog. They are not rows in the buyy `Product` table, they are not in the buyy bag, and Paystack does not charge them.

9.1 What the shop shows

The category row under the search has a link labeled **CJ specials**. It opens `/cj`. The homepage rail under the sale products is titled **cj special**. Cards show a black `cj` badge and a US dollar price. Opening a card goes to `/cj/:id`. That page does not add the item to the buyy bag.

9.2 What is stored

Table	Contents
`CjAuth`	Access token, refresh token, both expiry times, and `openId`. Server only.
`CjProduct`	CJ `pid`, name, description, USD price, image URL, category name, stock, and tags. The tag list always includes `cj`.
`CjVariant`	CJ `vid`, `sku`, price, and option label.

The buyy shop, search, sales, and Paystack checkout keep reading `Product` only.

9.3 How a sync runs

The client in `server/lib/cj.js` talks only to `https://developers.cjdropshipping.com/api2.0/v1`.

1. `CJ_API_KEY` lives in `server/.env`. It is not committed and it is not sent to the browser.
2. The server exchanges that key for an access token, stores it on `CjAuth`, and refreshes from the expiry dates. It does not request a new token on every product call.
3. An admin opens `/admin/cj`, enters a keyword and a page count (1 to 5), and posts to `POST /api/admin/cj/sync`.

4. The server pages `product/listV2` at 20 products per page, then calls `product/query` and `product/stock/getInventoryByPid` for each id. It waits about one second between CJ calls.

5. Rows are upserted into `CjProduct` and `CjVariant`.

The storefront reads `GET /api/cj/products` and `GET /api/cj/products/:id` from that local table. A page view does not call CJ. Admin status is `GET /api/admin/cj/status`. It reports whether a key is configured and whether a token is still active. It does not return the key, the token, or `openId`.

9.4 What this pass does not do

Checkout still charges only buyy products through Paystack. No CJ order is created, no CJ balance is spent, and no webhook is registered. Those steps stay in the connection guide until a public HTTPS callback exists and a sandbox order has been proved.